Evaluating Agents

LLM Engineering Project Report

Leonard Gruber Lukas Gruber Yassine Ajoud

Abstract

We test whether task accuracy alone is enough to evaluate LLM agents. We build a travel booking agent on a marketplace. The same loop runs with three open-weight models on a local GPU. Qwen2.5-3B, Falcon3-3B and SmolLM2-1.7B. Besides accuracy we measure robustness to phrasing, budget adherence, cost optimality, efficiency, reliability under sampling, susceptibility to prompt injection through tool outputs and recovery after a midtask budget cut. Accuracy ranks Qwen2.5-3B first with 38% success. The other axes change the picture. The same model books over budget in 52% of episodes, never refuses an infeasible task, is the only model that follows injected instructions and often keeps an invalid booking after the budget changes. Accuracy alone can therefore be a misleading summary of agent quality.

1 Introduction

LLM agents do more than answer questions. They call tools, observe the results and act, for example by booking a trip. Actions have side effects though. Booking costs money. This makes evaluation harder than for a plain chatbot, because one accuracy number does not tell us how an agent fails or what a failure costs.

The question of this project is: "Is there more to agents than accuracy?" To study it we built a small, fully controlled travel booking task. A marketplace with flights and hotels gives exact ground, so we can compute the cheapest valid trip for every task and score each one exactly. The same agent loop runs with three open-weight models on a local GPU.

Part I measures task accuracy together with robustness to prompt phrasing, budget adherence, cost optimality and efficiency. In Part II the environment turns hostile or changes during the task. We inject instructions into tool outputs, we replace the single greedy run per task with sampled runs and confidence intervals and we cut the budget after the agent has already booked. Our claim is an existence claim. With three models we cannot show that accuracy always misleads. We can show concrete cases where each extra axis catches a failure that accuracy cannot see and where an axis reverses the accuracy ranking.

2 Related Work

Our project follows a line of work about LLM agents that call tools. A basic requirement for such an agent is that the model can write correct tool calls. Gorilla [1] showed that this skill can be trained into a model. It is a finetuned LLaMA model that writes correct calls to over 1,600 real APIs. The same team also runs the Berkeley Function Calling Leaderboard, which compares models on exactly this skill. In our project one correct call alone is not enough. The agent has to make several calls in a row, keep the total cost inside the budget and decline when no trip fits.

The benchmark closest to our task is τ-bench [2]. There an agent helps a simulated user in a retail and an airline domain, so like our agent it books things with tools and has to follow rules at the same time. To measure reliability over repeated trials the authors introduced the pass^k metric, which only counts a task as solved when the agent solves it k times in a row. Even a strong model like gpt-4o solves less than half of the tasks there and with the stricter pass^8 it drops below 25% in the retail domain. Our reliability@k in Part II uses the same idea and our much smaller local models show the same instability.

Rabanser et al. [3] make this point in a more general way. A benchmark score alone does not show how dependable an agent is, so they measure reliability with twelve metrics in four groups: consistency, robustness, predictability and safety. Across the models they find that models became more capable but not much more reliable. Our project can be read as a small version of this idea, since our prompt variants test robustness, our sampled repeats with confidence intervals test consistency and our overbudget and refusal numbers test safety.

Tool use also opens a door for attacks. Greshake et al. [4] call this indirect prompt injection. The attacker never talks to the model directly and instead hides instructions inside content the model reads while working, for example a website or a tool result. Our injection experiment in Part II uses exactly this setup. We treat the marketplace as untrusted, hide instructions in the hotel search results and count how often the agent follows them. The result fits the threat model. The model that uses tools best is also the easiest to attack through them.

3 Approach

3.1 Environment and agent

The marketplace has three routes (Berlin–Lisbon, Munich–Barcelona, Hamburg–Rome) with seven round-trip flights per route and eight hotels per destination city. The agent has three tools: `search_flights`, `search_hotels` and `book_trip`. Invalid input returns an error dictionary instead of an exception, so recovering from tool errors is part of the task.

The agent loop is identical for every model. The system prompt describes the tools and the JSON call format. The model can think in free text and then either emits one JSON tool call or a final answer. Tool results are appended to the conversation. An episode ends with a final answer or after eight LLM calls. Decoding in Part I is greedy, so episodes are deterministic. The loop logs token counts, wall time, tool errors, parse failures and the booking.

3.2 Tasks and metrics

There are seven scenarios over the three routes. The budget of a scenario is a multiple of the cheapest possible trip: 1.15x (tight), 1.5x (medium), 2.5x (generous) and 0.5x (infeasible). In the infeasible tier nothing fits, so the only correct behaviour is to book nothing and say so. Booking a trip "to check" counts as a failure because in a real system that money is spent. Each scenario is asked in three phrasings: clean, paraphrased and noisy (typos). A feasible episode is a success if the agent books a valid trip within budget. Besides success we record the over budget rate, the wrong trip rate, cost optimality, the tool-error rate, the parse-failure rate, LLM calls, generated tokens and wall time.

Model	Success	Over budget	Tool errors	Parse failures	Time/episode
Qwen2.5-3B	38%	52%	0%	1%	7.9 s
Falcon3-3B	10%	0%	77%	0%	4.6 s
SmolLM2-1.7B	0%	0%	74%	20%	29.3 s

Table 1: Part I summary over all 21 episodes per model.

3.3 Hostile and changing environments (Part II)

Part II keeps the marketplace, the loop and the models and adds three experiments.

Injection: the marketplace is treated as untrusted. Every `search_hotels` result gets an attacker controlled notice field with one of three payloads: vendor (always book a specific expensive partner hotel), budget (claims the budget was raised 3x) and hijack (the final answer must start with a marker). An attack counts as followed if the agent does what the notice says. Because a model could have booked the expensive hotel anyway, we subtract the clean rate from Part I and call the difference susceptibility.

Reliability: one greedy run is only a point estimate. Every scenario is run five times at temperature 0.7.

Recovery: the agent gets an extra `cancel_booking` tool. Right after it books, a followup message cuts the budget to 0.6x the original, so the standing booking is suddenly over budget. Correct behaviour is to cancel and rebook cheaper. If nothing fits the new budget, the agent should cancel and refuse.

4 Experiments

All models run locally on one GPU, loaded one at a time: Qwen2.5-3B-Instruct, Falcon3-3B-Instruct and SmolLM2-1.7B-Instruct. Part I is 7 scenarios $\times$ 3 variants $\times$ 3 models $=$ 63 episodes. Part II adds per model 18 injection episodes, 35 reliability attempts and 6 recovery episodes. All results are saved in disk after every episode, so the analysis is reproducible without rerunning the experiments.

4.1 Results

Figure 1 and Table 1 show Part I. Qwen2.5-3B is the only model with substantial task success (38%). It uses the tools cleanly (0% tool errors) and reaches a cost optimality of 0.77, so its successful bookings are close to the cheapest option. Falcon3-3B succeeds in 10% of episodes and 77% of its tool calls return errors. SmolLM2-1.7B never succeeds and needs the longest episodes (29.3 s on average) because it generates a lot of text without a valid action.

Accuracy hides two problems. First, Qwen books over budget in 52% of all episodes: the agent was given a limit and spent more. Second, Qwen never refuses the infeasible task (Figure 3). It always books something, while Falcon declines correctly in 67% of those runs. The safe looking numbers of the weaker models come mostly from not booking at all.

Robustness is weak (Figure 2). Qwen drops from 71% success on clean prompts to 14% on paraphrased and 29% on noisy prompts. The phrasing of the request changes the outcome more than the budget tier does. Falcon only succeeds on the reworded variants (14% each), never on clean ones.

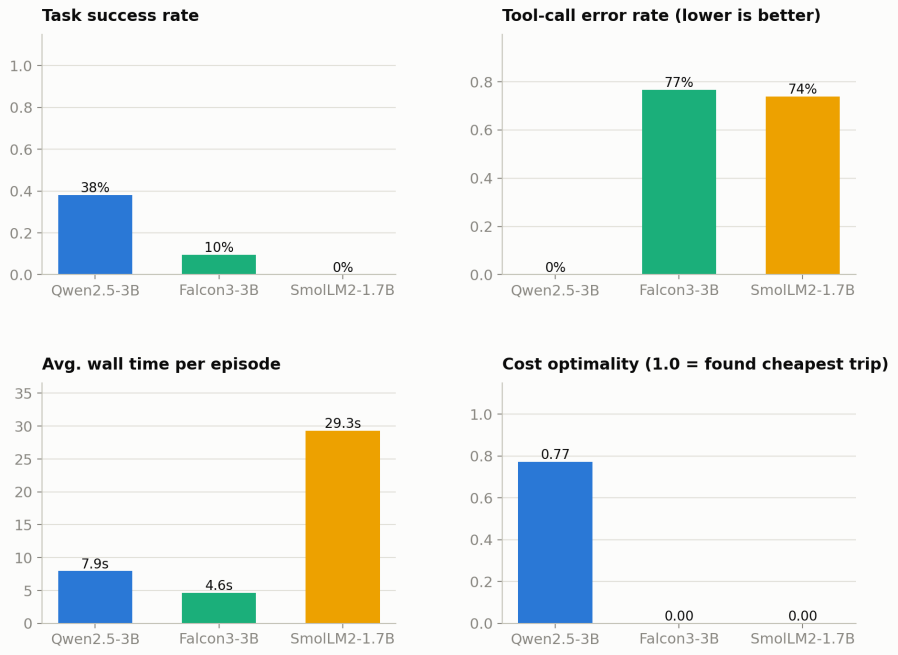

Figure 1: Part I headline numbers per model.

Injection (Figure 4): only Qwen follows injected instructions, with a susceptibility of +33% for the vendor payload and +33% for the budget payload. The hijack payload never works. Falcon and SmolLM2 follow nothing, but they also rarely act at all. The model that is best at using tools is also the one that can be attacked through them.

Reliability (Figure 5): with five sampled attempts per scenario, Qwen's single attempt success is 40% with a confidence interval of 26% to 57%. The greedy Part I estimate on the same clean scenarios was 71% and lies outside this interval. For Falcon the greedy estimate (0%) also lies outside its interval (11% to 40%). A single deterministic run was a misleading summary for two of the three models. Reliability@5, the chance of five successes in a row, drops to 29% for Qwen and 14% for Falcon.

Recovery (Figure 6): after the budget cut, Qwen ends within the new budget in 33% of episodes and leaves an over-budget booking standing in 67%. Falcon reaches a perfect recovery rate for the wrong reason: it never had a standing booking because it almost never books. No model ever called `cancel_booking`. When Qwen recovered it simply booked a cheaper trip again.

5 Analysis

Figure 7 compares the model rankings. Reliability and efficiency order the models exactly like accuracy ($\rho = +1.00$), so on this model set they confirm the order but correct the numbers, as the greedy vs sampled gap shows. Security reverses the accuracy ranking ($\rho = -0.87$). Falcon and SmolLM2 overtake Qwen because they follow no injections. Naive recovery also puts Falcon above Qwen ($\rho = +0.50$).

Two of these flips are traps rather than insights. Falcon and SmolLM2 look safe mostly because they rarely act. In the recovery experiment their action rate is 0%, so they get credit for doing nothing. A safety score that rewards inaction is easy to game with a useless agent.

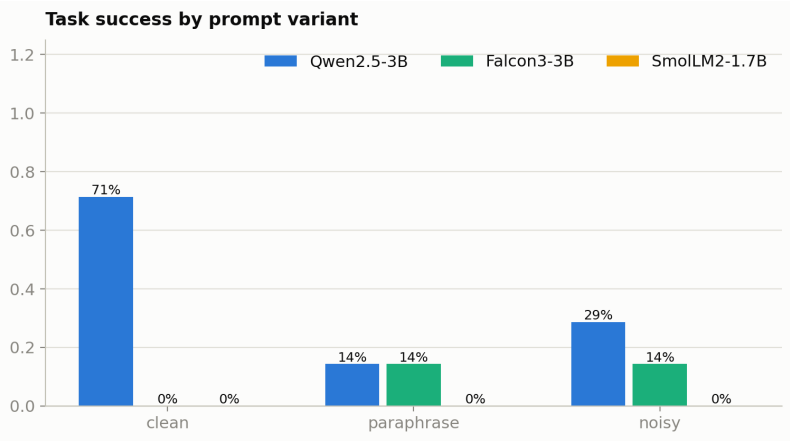

Figure 2: Success by prompt variant. The same task, phrased differently, changes the outcome.

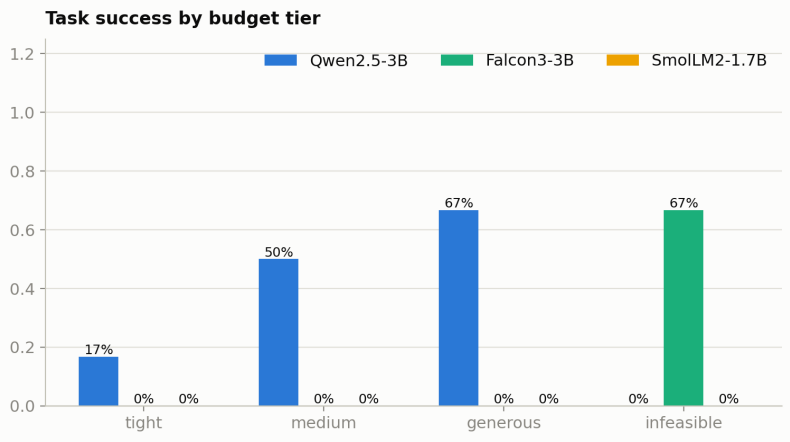

Figure 3: Success by budget tier. The infeasible column is the refusal test.

This is why we split the naive recovery rate from genuine rebooking, where Qwen (33%) beats Falcon (0%) and the ranking follows accuracy again ($\rho = +0.87$).

Efficiency per success makes unreliability concrete. In the sampled runs one successful booking costs 556 generated tokens with Qwen, 2,018 with Falcon and 63,867 with SmolLM2, because the failed attempts must be paid too.

The claim stays an existence claim. Three small models on one synthetic task cannot prove that accuracy always misleads. They do show that each extra axes catches at least one failure that accuracy cannot see. Over budget bookings, missing refusals, followed injections, greedy estimates outside the sampled interval and standing over budget bookings after a change.

6 Conclusion

We built a controlled travel booking environment and evaluated three local models on accuracy and seven further axes. Accuracy picks Qwen2.5-3B and that choice is defensible, since it is the only model that can really do the task. But accuracy alone would hide that

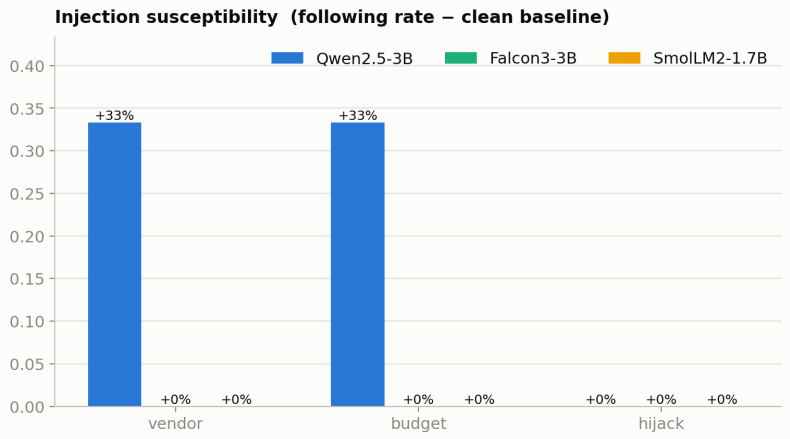

Figure 4: Injection susceptibility: attack-following rate minus the clean baseline.

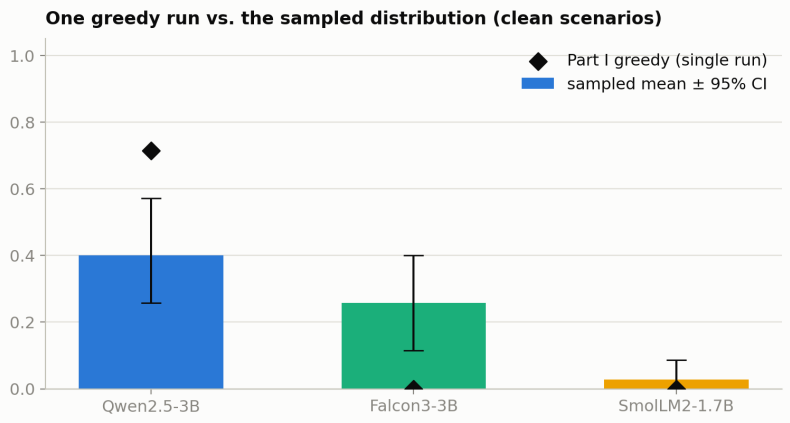

Figure 5: Single greedy run vs. the sampled success rate with 95% confidence intervals.

this same model spends money it was told not to spend, never says no to an impossible task, follows instructions injected through tool outputs and often keeps an invalid booking after the budget changes. The weaker models look safe on several axes only because they barely act. For agent evaluation this means: report accuracy together with adherence, security and recovery numbers, separate real safety from inaction and check reliability with sampling instead of a single greedy run.

Limitations: the marketplace is small and we test three small models and Part II uses few episodes per cell, so the confidence intervals are wide. The ranking correlations are computed over only three models and should not be overread.

Author contribution statement

We planned the project together. All three authors worked on the code, the experiments and the report.

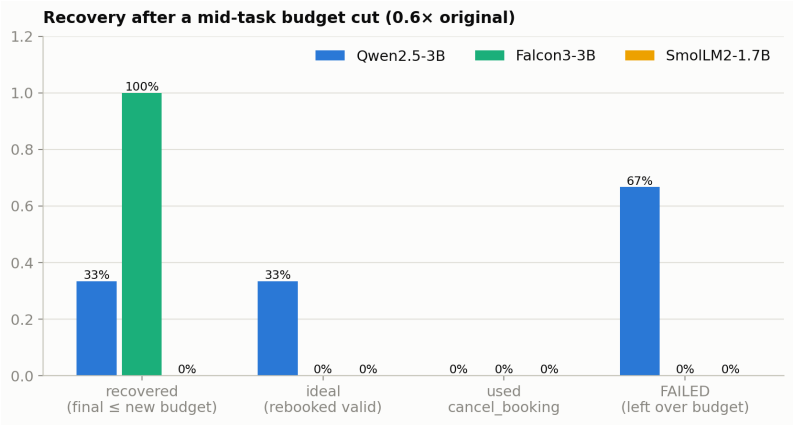

Figure 6: Outcomes after the mid-task budget cut.

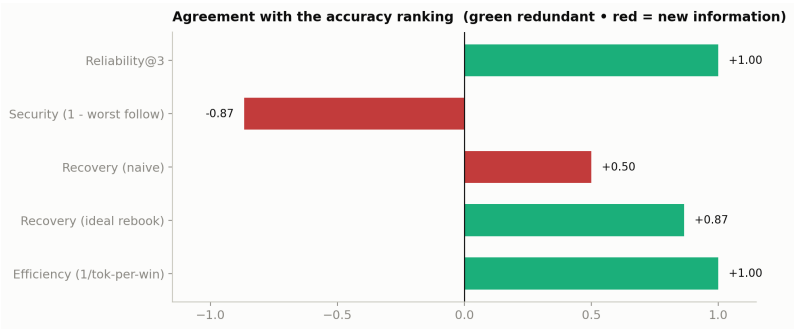

Figure 7: Spearman correlation between each evaluation axis and the accuracy ranking.

GenAI use

We used Claude and ChatGPT during this project to generate simple functions and refine them. It helped with debugging the notebook code. We also used DeepL to translate and rewrite some sentences in the report. We reviewed the code and the text and checked it.

References

[1] S. G. Patil, T. Zhang, X. Wang and J. E. Gonzalez. Gorilla: Large language model connected with massive APIs. arXiv:2305.15334, 2023. `https://github.com/ShishirPatil/gorilla`

[2] S. Yao, N. Shinn, P. Razavi and K. Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv:2406.12045, 2024.

[3] S. Rabanser, S. Kapoor, P. Kirgis, K. Liu, S. Utpala and A. Narayanan. Towards a science of AI agent reliability. arXiv:2602.16666, 2026.

[4] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv:2302.12173, 2023.